NEXUS Programming Language

Documentation

fanxcz

Copyright © 2026 fanxcz

Table of contents

1. Home 5
1.1 NEXUS 5
2. Downloads 6
2.1 Downloads 6
3. Screenshots 11
3.1 Screenshots and demos 11
4. Getting Started 12
4.1 Overview 12
4.2 Installation 14
4.3 Quick Start 17
4.4 Project Structure 20
5. Language Reference 0
5.1 Overview 0
5.2 Language Basics 0
5.3 Syntax 0
5.4 Types 0
5.5 Functions 0
5.6 Control Flow 0
5.7 Modules & Imports 0
5.8 Error Handling 0
5.9 FFI 0
6. Standard Library 0
6.1 Overview 0
6.2 Module Overview 0
6.3 std.io 0
6.4 std.fs 0
6.5 std.net 0
6.6 std.json 0
6.7 std.db 0
6.8 std.memory 0
6.9 std.crypto 0
6.10 std.os 0
6.11 std.string 0
7. Runtime 0
7.1 Overview 0

Copyright © 2026 fanxcz

7.2 Runtime Overview 0
7.3 Memory 0
7.4 Graphics 2D 0
7.5 Graphics 3D 0
7.6 Audio 0
7.7 Input 0
7.8 ECS 0
7.9 Scenes 0
7.10 Physics 0
7.11 Particles 0
7.12 UI 0
8. Tooling 0
8.1 Overview 0
8.2 Commands 0
8.3 Native backend 0
8.4 Backend selection 0
8.5 Formatter 0
8.6 Linter 0
8.7 Documentation Generator 0
8.8 Benchmarking 0
8.9 LSP 0
8.10 Editor 0
8.11 VS Code Extension 0
9. Package Manager 0
9.1 Overview 0
9.2 Manager Overview 0
9.3 Creating Packages 0
9.4 Using Packages 0
9.5 Publishing 0
10. Templates 0
10.1 Overview 0
10.2 App 0
10.3 Game 0
10.4 Paint 0
10.5 CLI 0
10.6 Library 0
10.7 Editor 0
10.8 Server 0

Copyright © 2026 fanxcz

10.9 3D 0
11. Examples 0
11.1 Index 0
11.2 Catalog 0
11.3 Games 0
11.4 Graphics 0
11.5 Audio 0
12. Build System 0
12.1 Overview 0
12.2 CMake Integration 0
12.3 Cross-compilation 0
12.4 Targets 0
13. Architecture 0
13.1 Overview 0
13.2 Repository Layout 0
13.3 Compiler Pipeline 0
13.4 Runtime Design 0
13.5 Memory Model 0
14. Development 0
14.1 Overview 0
14.2 Contributing 0
14.3 Building from Source 0
14.4 Testing 0
14.5 Platform Support Matrix 0
14.6 Debugging 0
14.7 Roadmap 0

Copyright © 2026 fanxcz

1. Home

1.1 NEXUS

Компилируемый язык программирования и нативный рантайм для приложений, инструментов и игр. Компилятор написан на C++20, часть экосистемы уже переписывается на самом NEXUS (`self-hosting`).

```
fn main() {
    let tool = "NEXUS"
    print("Hello from " + tool + "!")
}
```

```
nexus new MyApp --template cli
nexus build
nexus run
```

MIT · Windows · Linux · macOS

1.1.1 Документация

- Быстрый старт
- Синтаксис языка
- Стандартная библиотека
- Команды CLI
- Дорожная карта и self-hosting
- Репозиторий на GitHub

1.1.2 Контакты

Автор: fanxcz

- Telegram: @fanxcz
- Почта: sad098184@gmail.com

Баг-репорты и предложения — в issues на GitHub

October 24, 2026

October 24, 2026

fanxcz

Copyright © 2026 fanxcz

2. Downloads

2.1 Downloads

Download the complete documentation as PDF

Choose the official NEXUS package for your system. The Windows x64 package is assembled and tested by GitHub Actions from this repository. Downloads are published to GitHub Pages only when the packaging and documentation workflow succeeds.

Important for direct downloads: use the published URL below, not `raw.githubusercontent.com/.../docs/assets/downloads/NEXUS_Windows_Portable.zip`. The source tree previously contained an old, incomplete ZIP at that path; it is not the current package built by CI.

Copyright © 2026 fanxcz

Direct download of the current portable ZIP · Matching SHA-256

Copyright © 2026 fanxcz

:material-microsoft-windows: **NEXUS Setup for Windows 10/11 x64**

Recommended for most Windows users. Installs NEXUS for the current user, offers Start menu and desktop shortcuts, and can add the launcher to the user PATH. No administrator rights are required.

Download NEXUS-Setup-1.1.0.exe

Installer SHA-256

:material-archive: **Portable Windows x64 package**

Includes the compiler launcher and executable, bundled Zig-based C++ toolchain, runtime DLLs, standard library, templates, examples, and helper scripts. Extract to a permanent folder before running it.

Download Windows ZIP

ZIP SHA-256

:material-code-tags: **NEXUS language extension for VS Code**

Syntax highlighting, snippets, diagnostics, and completion for `.nx` source files.

Download VSIX

VSIX SHA-256

:material-monitor: **NEXUS Studio — Windows x64 desktop IDE**

Standalone desktop editor with a project explorer, `.nx` editor, compiler selection, Build/Run buttons, and output terminal. Install the NEXUS compiler separately and select it inside Studio.

Download NEXUS Studio Windows EXE

Studio installer SHA-256

:material-palette: **NEXUS Paint — native drawing app for Windows x64**

A desktop raster editor written in the NEXUS language. It includes a brush, eraser, line and rectangle tools, fill, colour palette, adjustable brush size, undo/redo and BMP save/load. The portable ZIP is recommended because it bundles the SDL runtime DLLs and editable `.nx` source.

Download NEXUS Paint portable ZIP

Direct ZIP download from the repository

Download EXE only (requires DLLs from the ZIP)

EXE SHA-256 · ZIP SHA-256

View the NEXUS source

:material-laptop: **Linux and macOS compiler builds**

Linux x86-64 and macOS Apple Silicon/Intel archives are built by the tagged-release workflow. They become downloadable when a version-tag release has completed successfully; check the official Releases page for currently published assets.

Copyright © 2026 fanxcz

Browse GitHub Releases

:material-source-repository: **Source code**

Compiler, standard library, runtime, tests, examples, templates, and documentation.

Open the repository

2.1.1 Install on Windows

1. Download `NEXUS-Setup-1.1.0.exe` from this page after the artifact is available.
2. Run the installer and follow the prompts. It installs under `%LOCALAPPDATA%\Programs\NEXUS` by default.
3. Select the optional shortcuts and PATH integration you want.
4. Close and reopen PowerShell, then verify:

```
nexus --version
nexus doctor
nexus self-test
```

If the command is not found, first open a new terminal. You can test the installed launcher directly:

```
& "$env:LOCALAPPDATA\Programs\NEXUS\nexus.cmd" --version
```

The installer is not code-signed. Only run it if you downloaded it from this repository's official GitHub Pages output or GitHub Actions artifact and are comfortable with the unsigned-app warning Windows may display.

2.1.2 Verify and unpack the current Windows portable package with PowerShell

The ZIP and its checksum must both be downloaded from the published Pages URL. This command validates the hash and confirms the archive really contains `nexus.cmd` before you install it.

```
$ErrorActionPreference = 'Stop'
$base = 'https://fanxcz.github.io/nexus-lang/assets/downloads/'
$work = Join-Path $env:TEMP 'NEXUS-Windows-Portable'
New-Item -ItemType Directory -Path $work -Force | Out-Null
$zip = Join-Path $work 'NEXUS_Windows_Portable.zip'
$sum = Join-Path $work 'NEXUS_Windows_Portable.zip.sha256.txt'
$stage = Join-Path $work 'unpacked'

Invoke-WebRequest ($base + 'NEXUS_Windows_Portable.zip') -OutFile $zip
Invoke-WebRequest ($base + 'NEXUS_Windows_Portable.zip.sha256.txt') -OutFile $sum
$expected = (Get-Content $sum -Raw).Trim() -split '\s+' | 0 | ToLowerInvariant()
$actual = (Get-FileHash $zip -Algorithm SHA256).Hash.ToLowerInvariant()
if ($actual -ne $expected) { throw 'SHA-256 mismatch; archive rejected.' }

if (Test-Path $stage) { Remove-Item $stage -Recurse -Force }
Expand-Archive -LiteralPath $zip -DestinationPath $stage -Force
$launcher = Get-ChildItem -LiteralPath $stage -Filter 'nexus.cmd' -File -Recurse | Select-Object -First 1
if (-not $launcher) { throw 'The published ZIP does not contain nexus.cmd. Do not install it.' }
Write-Host "Verified launcher: $($launcher.FullName)"
```

2.1.3 Portable installation

1. Extract the ZIP to a permanent writable folder, for example `C:\NEXUS`.
2. Double-click `Install-NEXUS.bat` if you want to add that folder to your user PATH.
3. Open a new terminal and run `nexus --version`, `nexus doctor`, or `nexus self-test`.

The portable package is designed to avoid requiring a separately installed LLVM/Clang toolchain for its bundled Windows workflow. Feature availability still depends on the selected compiler backend and target.

Copyright © 2026 fanxcz

2.1.4 Integrity

Use PowerShell to calculate the checksum of a downloaded file:

```
Get-FileHash .\NEXUS-Setup-1.1.0.exe -Algorithm SHA256
Get-FileHash .\NEXUS_Windows_Portable.zip -Algorithm SHA256
```

Compare each hash with the matching `.sha256.txt` file published alongside the download.

October 25, 2026

October 24, 2026

fanxcz

Copyright © 2026 fanxcz

3. Screenshots

3.1 Screenshots and demos

The current source archive does not contain verified application screenshots. Rather than display broken or misleading images, this page records the visual demos available in the repository.

3.1.1 Included demos

- **Nexus Paint** — raster canvas example with drawing tools.
- **Neon Survivor** — wave-based survival game example.
- **Nexus Arena** — arena-style game example.
- **3D demo** — native 3D rendering example.
- **Terminal game** — text-based game example.

Open the source files in the [redacted] directory. Screenshots will be linked here after captures from real runs are added to the repository.

October 24, 2026

October 24, 2026

fanxcz

Copyright © 2026 fanxcz

4. Getting Started

4.1 Overview

4.1.1 Getting Started

Welcome to NEXUS! This section will help you install NEXUS and create your first project.

In This Section

Guide	Description
Installation	Install NEXUS on your platform
Quick Start	Create your first project in 5 minutes
Project Structure	Understand the project layout

Prerequisites

- **OS**: Linux (x86_64), Windows (x86_64), macOS (x86_64/arm64)
- **For prebuilt Windows packages**: Windows 10/11 x64; download the tested installer from **Downloads**.
- **For source builds**: C++20 compiler, CMake 3.25+ for the checked-in presets, and Ninja. SDL2 development libraries are needed for graphics/runtime builds.

Install from a prebuilt package

For Windows, use the **NEXUS Setup installer or portable ZIP**. The installer is the recommended option for most users; the portable ZIP is useful when you want to choose its location yourself.

Build from source on Linux

```
sudo apt-get install cmake ninja-build clang-18 lld-18 libsdl2-dev

git clone https://github.com/fanxcz/nexus-lang.git
cd nexus-lang
cmake --preset release
cmake --build --preset release
sudo cmake --install build
```

The compiler's built-in native backend supports a defined language subset without a separate Clang toolchain. Broader LLVM-based builds may need Clang/LLVM, and graphics/runtime features may need SDL2 libraries.

Verify Installation

```
nexus --version
# nexus 1.1.0
```

Next Steps

1. Follow the **Quick Start** to create your first project
2. Learn the **Language Basics**
3. Explore the **Standard Library**

Copyright © 2026 fanxcz

4. Check out **Examples**

October 24, 2026

October 24, 2026

fanxcz

Copyright © 2026 fanxcz

4.2 Installation

4.2.1 Installation

Pre-built Binaries

Windows builds are assembled by the **Downloads workflow**. Linux and macOS archives are published to **GitHub Releases** when a version tag is built successfully. If no tagged release is listed yet, use the source-build instructions below or the latest successful CI artifact while it is available.

LINUX (X86_64)

Portable and native compiler artifacts are provided through project releases when available. The built-in machine-code backend can produce x86-64 Linux ELF executables for its supported language subset; programs outside that subset need the configured LLVM/Clang fallback.

WINDOWS 10/11 (X86_64) — SELF-CONTAINED PORTABLE BUNDLE

Download the **NEXUS Windows portable ZIP** and extract the entire archive to a folder such as `C:\NEXUS`. The bundle is designed to include the NEXUS compiler, its pinned fallback toolchain and target files, the NEXUS standard library, templates, runtime assets, and SQLite support.

For normal use with the supported self-contained Windows package, you should not need to separately install LLVM/Clang or Visual Studio C++ Build Tools. The compiler's built-in native backend handles its documented x86-64 subset; other programs use the bundled Zig-backed C++ toolchain. Building NEXUS itself from source still requires a C++20 toolchain and CMake. Run `check-NEXUS-toolchain.bat` to verify the included backend can compile and launch a native program, then use the terminal commands below. Keep the `bin`, `toolchain`, `runtime`, `stdlib`, and `project-templates` folders together.

To add NEXUS to your user PATH:

1. Run `install-NEXUS.bat` from the extracted folder. It checks the bundled backend and adds the folder to your user PATH.
2. Close and reopen PowerShell, Windows Terminal, or your editor terminal.
3. Verify the command:

```
nexus --version
nexus doctor
```

1. Create or run a project:

```
nexus new HelloNexus --template cli
cd HelloNexus
nexus run
```

The PATH helper changes only the current user's PATH and does not require administrator privileges. Keep the extracted folder in the same location after adding it to PATH; run `Uninstall-NEXUS-Path.ps1` before deleting that folder.

MACOS (X86_64 / ARM64)

Use the project release artifacts when published for your architecture. The native machine-code emitter is currently focused on x86-64 Windows/Linux. macOS builds use the configured Clang toolchain as required by the program and target.

Copyright © 2026 fanxcz

Building from Source

The compiler's `CMakeLists.txt` minimum is 3.20, but the checked-in `CMakePresets.json` uses schema version 6, which requires CMake 3.25+. If you have CMake 3.20–3.24, use explicit configure/build commands (`cmake -S . -B build -G Ninja -DNEXUS_BUILD_TESTS=ON` and `cmake --build build`) instead of `--preset` commands.

PREREQUISITES

Dependency	Minimum Version	Notes
CMake	3.20	Required for build system
Ninja	1.10	Recommended generator
Clang/LLVM	18	Required for LLVM fallback builds; not needed for all built-in native subset programs
SDL2	2.28	Optional for building the graphics/audio-enabled runtime
C++20 Compiler	Clang 18 / GCC 13 / MSVC 19.40	

Ubuntu / Debian

```
sudo apt-get update && sudo apt-get install -y \
    cmake ninja-build clang-18 lld-18 \
    libsdl2-dev libsdl2-image-dev libsdl2-mixer-dev libsdl2-ttf-dev
```

Arch Linux

```
sudo pacman -S cmake ninja clang lld sdl2 sdl2_image sdl2_mixer sdl2_ttf
```

Fedora

```
sudo dnf install cmake ninja-build clang lld SDL2-devel SDL2_image-devel SDL2_mixer-devel SDL2_ttf-devel
```

Windows (PowerShell)

```
choco install cmake ninja llvm sdl2
# Or use vcpkg: vcpkg install sdl2 sdl2-image sdl2-mixer sdl2-ttf
```

macOS

```
brew install cmake ninja llvm sdl2 sdl2_image sdl2_mixer sdl2_ttf
```

BUILD STEPS

```
git clone https://github.com/fanxcz/nexus-lang.git
cd nexus-lang

cmake --preset release
cmake --build --preset release

# Unit, integration, backend, LSP, doctor, and self-test checks
ctest --test-dir build --output-on-failure

# Diagnose the local environment
./build/nexus doctor --json

# Install (optional)
sudo cmake --install build
```

VERIFY INSTALLATION

```
nexus --version
nexus doctor
nexus self-test
```


Copyright © 2026 fanxcz

IDE Integration

VS CODE

Follow the VS Code language extension guide. The project contains the extension source but does not yet publish a Marketplace extension. For basic workspace settings and build task generation, run `nexus editor` inside a NEXUS project. Configure `nexus lsp` as the language server for diagnostics and completion.

CLION

Open the NEXUS compiler repository and select a supported configure preset from `CMakePresets.json`.

NEOVIM / VIM

The NEXUS language server can be configured to run `nexus lsp` using the editor's standard LSP client. Compiler C++ development can use `clangd` after generating a compilation database with CMake.

Troubleshooting

"CLANG NOT FOUND"

Run `nexus doctor` to see whether the LLVM fallback was found. Supported built-in native-subset programs may still compile without a Clang executable. Programs outside the built-in subset require an available compiler via PATH, `NEXUS_CXX`, or the toolchain supplied by the platform package.

SDL2 NOT FOUND

CMake can build the compiler without SDL2, but the graphics/audio runtime will be limited. Install SDL2 development libraries for your platform and reconfigure the project if you need those features.

GETTING HELP

- Check GitHub Issues for known problems.
- Run `nexus doctor --json` for script-friendly environment diagnostics.
- Open a new issue with a reproducible example.

October 24, 2026

October 24, 2026

fanxcz

Copyright © 2026 fanxcz

4.3 Quick Start

4.3.1 Quick Start

This guide uses the command-line workflow and runtime API included with NEXUS 1.1.0. Start with a small console program, then try a native graphical app.

1. Check the compiler

Open a terminal and run:

```
nexus --version
nexus doctor
```

On Windows, use PowerShell or Windows Terminal. If `nexus` is not found, follow the **installation guide**

INTERACTIVE CALCULATIONS

Run `nexus` with no arguments, or use `nexus repl` to evaluate expressions in the NEXUS language:

```
nexus> 2 + 2
4
nexus> let mut answer = 21
nexus> answer * 2
42
nexus> answer = answer + 1
nexus> answer
22
```

Use `:help` for commands, `:vars` to list saved bindings, `:clear` to reset the session and `:quit` to exit. This is a NEXUS evaluator, not a Python interpreter.

2. Create and run a command-line project

```
nexus new HelloNexus --template cli
cd HelloNexus
nexus check
nexus build
nexus run
```

NEXUS creates a project directory with a manifest and source file. Run commands from that project directory so the compiler can find its settings.

A small NEXUS program looks like this:

```
fn main() {
    print("Hello from NEXUS!")
}
```

3. Create a native graphical app

NEXUS includes an app template using the runtime's graphics API. Create it in a separate directory:

```
nexus new MyApp --template app
cd MyApp
nexus check
nexus build
nexus run
```

The template is a complete working example. Its core loop uses graphics functions present in the repository:

```
fn main() {
    let ok = gfx_init(1000, 640, "NEXUS App")
    if !ok {
```


Copyright © 2026 fanxcz

```
        print("Graphics backend unavailable: " + gfx_error())
        return
    }

    while !gfx_should_close() {
        gfx_poll()
        if gfx_key_down("ESC") {
            break
        }

        gfx_begin()
        gfx_clear(0.035, 0.045, 0.08, 1.0)
        gfx_rect(40.0, 40.0, 920.0, 110.0, 0.07, 0.12, 0.22, 1.0)
        gfx_text(70.0, 70.0, 3.0, "NEXUS APP", 0.55, 0.9, 1.0, 1.0)
        gfx_text(70.0, 120.0, 1.5, "ESC closes the window", 0.8, 0.84, 0.95, 1.0)
        gfx_end()
    }

    gfx_shutdown()
}
```

This example is based on `examples/templates/app.nx`. Graphics require the runtime dependencies supplied by the Windows portable package or installed for your platform/build.

4. Explore other templates

Template	Use it for
`cli`	Command-line utilities
`app`	Windowed native applications
`game`	2D game starting point
`3d`	3D graphics starting point
`paint`	Raster-canvas drawing application

| `server` | Server experiments | | `library` | Reusable source package |

The experimental editor demo is not a template accepted by `nexus new`. See **Editor source** for the tested way to try it. For example, `nexus new MyPaint --template paint` creates a Paint application scaffold. See **Templates** and **Examples** for details.

5. Useful commands

Command	Description
`nexus check`	Check the current project or a source file
`nexus build`	Build the current project
`nexus build --release`	Build an optimized project
`nexus run`	Build and run the current project
`nexus doctor`	Diagnose the compiler/runtime setup
`nexus self-test`	Compile and run the compiler's small self-test
`nexus explain-backend src/main.nx`	Explain which backend will be used

Use `nexus --help` to list the supported CLI commands. Command-specific `--help` flags are not currently implemented. The built-in native backend supports a defined x86-64 Linux/Windows subset; other programs may require the LLVM/Clang path. See **Backend selection**.

Copyright © 2026 fanxcz

Next steps

- Read the **Language Reference**.
- Explore the **Standard Library**.
- Learn the **Runtime API**.
- Browse **Examples**
- Read **Building from Source**.

October 24, 2026

October 24, 2026

fanxcz

Copyright © 2026 fanxcz

4.4 Project Structure

4.4.1 Project Structure

NEXUS keeps generated projects simple. The recommended layout has six top-level folders; only `src/` is required for a basic program. Add assets, tests, dependencies, build output, and documentation when the project needs them.

Standard Layout

```
project-root/
├── nexus.toml              # Project manifest
├── README.md               # Project overview
├── .gitignore              # Files excluded from version control
├── nexus.lock              # Optional dependency lockfile
├── src/                    # Required: source files
│   ├── main.nx             # Entry point (or [build].entry)
│   └── *.nx                # Additional modules
├── assets/                 # Optional: runtime assets
│   ├── images/
│   ├── audio/
│   ├── fonts/
│   └── shaders/
├── tests/                  # Optional: test programs
│   └── *.nx
├── vendor/                 # Optional: local packages (gitignored)
│   └── <package-name>/
│       ├── nexus.toml
│       └── src/
├── build/                  # Optional: generated output (gitignored)
└── docs/                   # Optional: project documentation
```

The six top-level folders are `src/`, `assets/`, `tests/`, `vendor/`, `build/`, and `docs/`. Start with `src/main.nx` and add the other folders only when needed.

`nexus.toml` Reference

`[package]` SECTION

```toml
[package]
name = "my-project"           # Required: Package name (lowercase, hyphens)
version = "1.0.0"             # Required: SemVer version
edition = "2026"              # Required: Language edition
description = "..."           # Optional: Short description
authors = ["Name <email>"]    # Optional: Author list
license = "MIT"               # Optional: SPDX license identifier
repository = "https://..."    # Optional: Repository URL
homepage = "https://..."      # Optional: Project homepage
keywords = ["game", "2d"]     # Optional: Search keywords
categories = ["games"]        # Optional: Categories
readme = "README.md"          # Optional: README path
```

`[build]` SECTION

```toml
[build]
entry = "src/main.nx"         # Entry point (default: src/main.nx)
optimization = "speed"        # "speed" | "size" | "debug" (default: speed)
debug_symbols = true          # Include debug info (default: true for debug)
target_cpu = "native"         # Target CPU (default: native)
linker = "lld"                # Linker: "lld" | "mold" | "gold" | "default"
rpath = true                  # Embed rpath (default: true)
```

`[targets]` SECTION

```toml
[targets]
default = "x86_64-linux"      # Default target triple

# Named targets for cross-compilation
windows = "x86_64-windows"
macos = "x86_64-macos"
linux_arm64 = "aarch64-linux"
web = "wasm32-unknown-unknown"  # Experimental
```


Copyright © 2026 fanxcz

`[dependencies]` SECTION

```
[dependencies]
# Local path dependency
my-lib = { path = "vendor/my-lib" }

# Git dependency (future)
# my-lib = { git = "https://github.com/user/repo", rev = "v1.0.0" }

# Registry dependency (future)
# my-lib = "1.2.3"
```

`[dev-dependencies]` SECTION

```
[dev-dependencies]
test-utils = { path = "vendor/test-utils" }
```

`[profile.*]` SECTIONS

```
[profile.debug]
optimization = "debug"
debug_symbols = true
overflow_checks = true

[profile.release]
optimization = "speed"
debug_symbols = false
lto = "thin"          # "off" | "thin" | "full"
strip = true
```

Source Files

MODULE STRUCTURE

```
src/
├── main.nx           # Main entry point
├── game/             # Game module
│   ├── mod.nx        # Module definition
│   ├── player.nx
│   └── enemy.nx
├── render/           # Render module
│   ├── mod.nx
│   └── sprite.nx
└── utils.nx          # Single-file module
```

MODULE DECLARATION (`mod.nx`)

```
// game/mod.nx
module game {
    export fn spawn_player(x: f32, y: f32) -> Entity
    export fn update_player(e: Entity, dt: f32)
    export struct Player { ... }
}
```

IMPORTING MODULES

```
// Import standard library
import std.io
import std.fs

// Import local module
import game
import render

// Import with alias
import std.net as net

// Selective import
import std.json { parse, stringify }
```

Assets

Assets in `assets/` are copied to the build output and accessible at runtime:

```
// Load image from assets/images/
let texture = gfx_load_texture("assets/images/player.png")

// Load sound from assets/audio/
```


Copyright © 2026 fanxcz

```
let sound = audio_load_sound("assets/audio/jump.wav")

// Load font from assets/fonts/
let font = gfx_load_font("assets/fonts/main.ttf", 24)
```

Use `nexus assets` command to manage assets:

```
nexus assets list          # List all assets
nexus assets add sprite.png # Add asset
nexus assets remove old.png # Remove asset
```

Build Output

The compiler writes generated artifacts into the project's `build/` directory (or the configured artifact path). A native build produces an executable and may keep the intermediate LLVM IR file used by the native backend. Windows executables use the `.exe` suffix.

```
build/
├── debug/                # Debug build artifacts (when using a project manifest)
├── release/              # Optimized build artifacts (when using --release)
└── <generated files>     # Native executable and intermediate IR
```

The compiler links the native runtime into generated applications; the source repository's `src/runtime/`, `stdlib/`, and `examples/templates/` directories are not copied into every project's build output.

Ignored Files

Add to `.gitignore`:

```
# Build output
build/
*.o
*.obj
*.exe
*.pdb

# Package cache
vendor/
nexus.lock

# IDE
.vscode/
.idea/
*.swp
*.swo

# OS
.DS_Store
Thumbs.db
```

Multi-Project Workspaces

For larger projects, use a workspace layout:

```
workspace/
├── nexus-workspace.toml   # Workspace config
├── engine/                # Engine library
│   ├── nexus.toml
│   └── src/
├── game/                  # Game executable
│   ├── nexus.toml
│   └── src/
└── tools/                 # Development tools
    ├── nexus.toml
    └── src/
```

Workspace config:

```
# nexus-workspace.toml
[workspace]
members = ["engine", "game", "tools"]
default_target = "x86_64-linux"
```


Copyright © 2026 fanxcz

October 24, 2026

October 24, 2026

fanxcz

Copyright © 2026 fanxcz